Національний університет «Одеська Політехніка»
Інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота №2
з дисципліни «Програмування мобільних пристроїв на базі ОС Android»
Тема: «Списки та управління станом у Jetpack Compose»

Виконав:
студент групи АІ-234
Середа Я. В.

Перевірили:
Годовиченко М. А.
Соловйова Д. В.

Одеса 2026

МЕТА РОБОТИ

Ознайомитися з принципами побудови динамічних списків за допомогою декларативного фреймворку Jetpack Compose. Засвоїти патерни управління станом з використанням архітектурного компонента ViewModel та реактивних потоків даних StateFlow. Здобути практичні навички реалізації односпрямованого потоку даних у мобільних застосунках.

ЗАВДАННЯ

- Розробити одноекранний Android-застосунок «Голосування» з динамічним списком на основі `LazyColumn`.
- Реалізувати можливість додавання нових варіантів через форму введення (`TextField` + кнопка).
- Налаштувати керування станом: зміна кількості голосів при натисканні на кнопку конкретної картки.
- Забезпечити автоматичне сортування списку за спаданням кількості голосів.
- Реалізувати відображення реактивного лічильника у заголовку, що показує поточного лідера голосування.

1 ХІД РОБОТИ

1.1 Створення моделі даних

Було створено `data class VoteOption` з полями `id` (з використанням `UUID`), `title` та `votes`. Використання `data class` забезпечило можливість імутабельного оновлення стану через метод `.copy()`.

1.2 Розробка ViewModel та налаштування StateFlow

Створено клас `VoteViewModel`. Локальний стан списку збережено у приватній змінній `MutableStateFlow`. Для UI відкрито дві публічні змінні `StateFlow`:

1) `sortedOptions`: автоматично сортує список за спаданням;
2) `leader`: динамічно визначає варіант з максимальною кількістю голосів.

Реалізовано методи `vote(id)` та `addOption(title)` з дотриманням принципів імутабельності (створення нових списків замість мутації існуючих).

1.3 Побудова UI-компонентів

Створено компонент `VoteCard` для відображення окремого варіанту.

Створено головний екран `VoteScreen`, що містить поле `OutlinedTextField` для введення, кнопку «Додати» та список `LazyColumn`.

Ключовим етапом стала прив'язка `key = { it.id }` у `LazyColumn` для коректного перемальовування UI під час автоматичного сортування елементів.

1.4 Прив'язка стану до інтерфейсу

За допомогою `collectAsState()` організовано підписку UI на зміни `StateFlow` з ViewModel.

2 ПЕРЕВІРКА ФУНКЦІОНАЛЬНОСТІ ДОДАТКУ

2.1 Перевірка початкового стану додатка

При першому запуску додатка відображається попередньо заданий список варіантів, відсортований за спаданням кількості голосів. Заголовок коректно визначає поточного лідера (рис. 2.1).

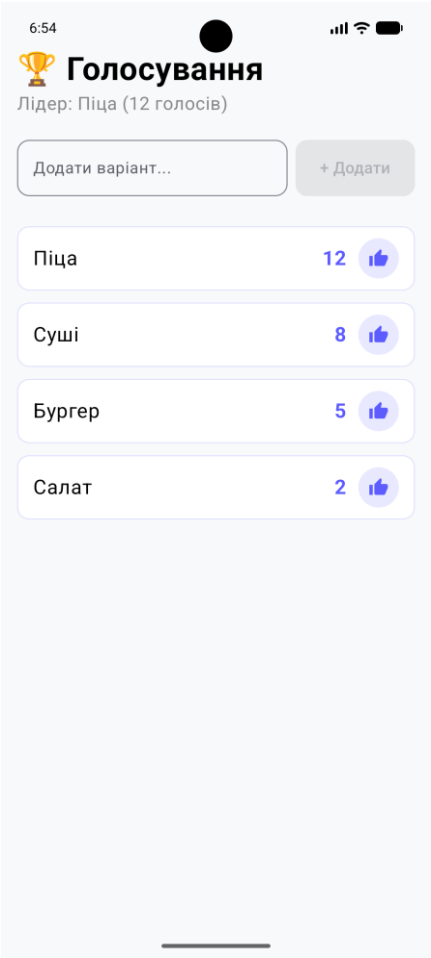

Рисунок 2.1 – Початковий стан екрана з відображенням відсортованого списку та лідера

2.2 Перевірка додавання нового варіанту

У текстове поле було введено новий варіант. Кнопка «Додати» стала активною лише після введення тексту. Після натискання новий варіант

з’явився у самому низу списку з нульовою кількістю голосів, а текстове поле автоматично очистилося (рис. 2.2).

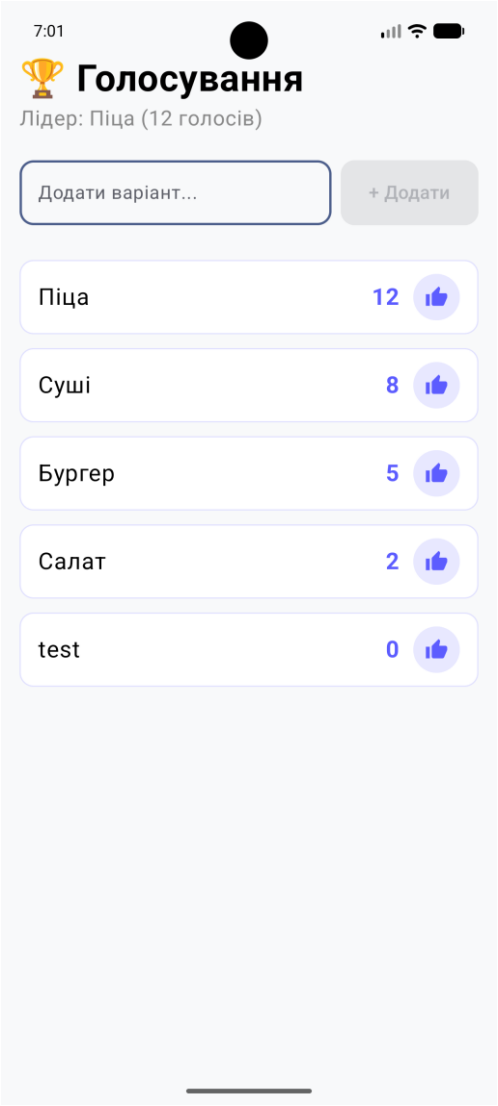

Рисунок 2.2 – Додавання нового варіанту з нульовою кількістю голосів

2.3 Перевірка логіки голосування

Після натискання на кнопку лайка біля варіанту з меншою кількістю голосів, лічильник цього варіанту миттєво збільшився на 1 (рис. 2.3).

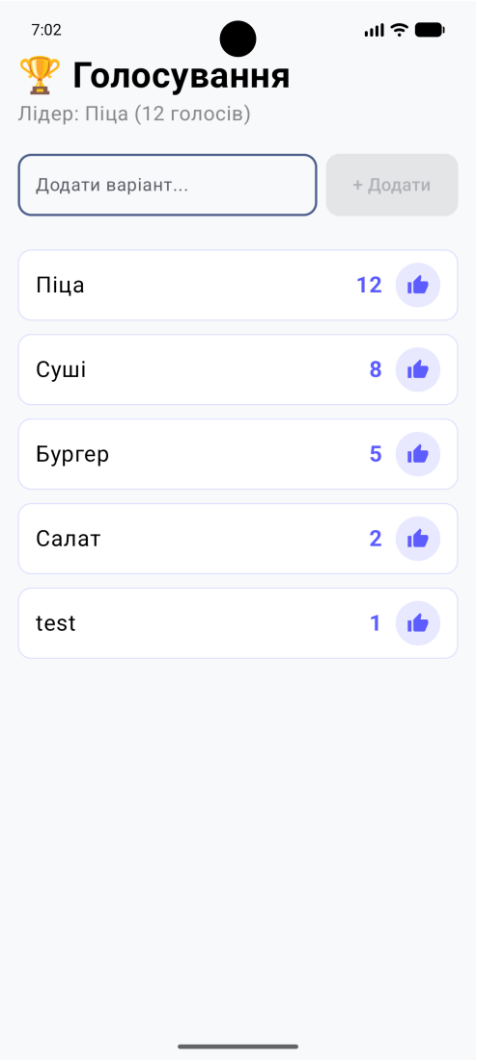

Рисунок 2.3 – Збільшення лічильника голосів після натискання на відповідну кнопку

2.4 Перевірка автоматичного сортування та зміни лідера

Було проведено серію натискань на кнопку голосування за варіант, який знаходився нижче у списку, доки його кількість голосів не перевищила лідера. Список автоматично відсортувався, а заголовок екрана миттєво оновився, відобразивши нового лідера та його кількість голосів (рис. 2.4).

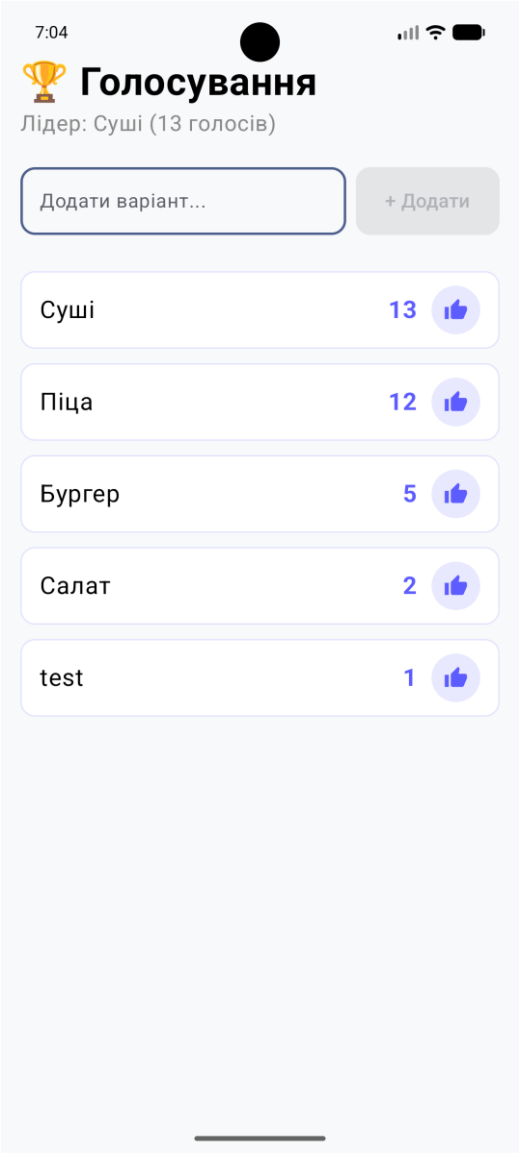

Рисунок 2.4 – Автоматичне пересортування списку та реактивна зміна лідера у заголовку

ВИСНОВОК

Під час виконання лабораторної роботи було успішно розроблено застосунок з динамічним списком. Практично закріплено роботу з архітектурним компонентом ViewModel, що дозволило відокремити бізнес-логіку від відображення. Завдяки використанню `StateFlow` та `collectAsState()` було реалізовано реактивний інтерфейс, який автоматично та коректно оновлюється при зміні даних (голосуванні, додаванні нових елементів, зміні лідера). Особливу увагу було приділено принципам імутабельності при роботі зі станом та правильному використанню ключів у `LazyColumn` для забезпечення плавних перестановок списку.